METODOLOGÍA I

Trabajo Práctico Integrador

Sistema odontológico: del proyecto fundado al sistema funcionando

OBJETIVO GENERAL

Continuar el sistema de gestión de turnos y agenda de pacientes para consultorios odontológicos iniciado en el Parcial 2. Integrar la fundación realizada con Active Stack, el desarrollo mediante Spec-Driven Development y el uso de una skill propia y un servidor MCP. Presentar un avance real, comprendido por el equipo y respaldado por pruebas.

La meta es completar el sistema previsto en el roadmap. Sin embargo, **no es obligatorio terminar todos sus changes para aprobar**: se admite un alcance parcial que cumpla el mínimo exigido y funcione hasta el punto alcanzado. Se consideran las distintas posibilidades de acceso a agentes de IA y sus límites de uso. La evaluación prioriza la calidad, las pruebas y la comprensión del trabajo realizado.

PUNTO DE PARTIDA Y MODALIDAD

Trabajen sobre el **mismo proyecto y repositorio del Parcial 2**, conservando su Discovery, base de conocimiento, roadmap e instrucciones del agente. Continúen con la modalidad y el equipo del trabajo anterior. La **nota del TPI es grupal** y todos los integrantes deben participar y explicar parte del trabajo en el video.

El TPI integra los trabajos de las pestañas **SDD + Active Stack** y **MCP + Skills**. Aunque la entrega independiente del segundo trabajo haya sido opcional, **la creación e integración de una skill propia, la conexión de un MCP y la explicación de su aplicación son obligatorias en este TPI**.

1. ALCANCE MÍNIMO OBLIGATORIO

El equipo debe presentar, como mínimo, **dos changes relevantes, completos y archivados por cada integrante, calculados sobre el total del proyecto**. Se incluyen los changes ya terminados para el Parcial 2.

Integrantes	Mínimo total de changes válidos
1	2
2	4
3	6

La regla es **mínimo total = 2 × cantidad de integrantes**. Se aplica al total grupal: no es necesario asignar dos changes a cada persona ni identificar un autor exclusivo de cada change. Esto no reemplaza la participación de todos en la explicación y validación del proyecto.

Presentar menos changes válidos que ese mínimo implica no aprobar el TPI. Cumplir la cantidad es una condición necesaria; también deben cumplirse los requisitos de funcionamiento, verificación, explicación y entrega de esta consigna.

Qué se entiende por un change relevante

Debe representar un avance coherente y verificable del roadmap, con un propósito claro y un resultado útil para construir o utilizar el sistema. Su tamaño se justifica por el alcance y las dependencias del proyecto, no por la cantidad de archivos o líneas de código.

- **Fundación técnica ejecutable:** estructura de carpetas acompañada por la configuración necesaria, arranque de la aplicación y comprobaciones previstas para esa etapa.
- **Modelos y persistencia:** entidades y relaciones necesarias para el dominio, con su verificación correspondiente.
- **Autenticación y permisos:** inicio de sesión, acceso autorizado y rechazo de accesos inválidos, según la especificación.
- **Agenda y turnos:** disponibilidad, reserva, cancelación u otra funcionalidad del dominio definida en el proyecto.
- **Frontend integrado:** una interfaz que permita ejecutar el flujo previsto contra el backend implementado.

Son ejemplos orientativos. Cada equipo conserva su arquitectura, sus nombres y el orden de dependencias de su roadmap. No se exige una numeración común ni llegar a un change específico. **No cuentan divisiones artificiales de tareas menores**, como separar la creación de cada carpeta, cambiar textos o generar archivos vacíos únicamente para alcanzar la cantidad.

Cómo adaptar el roadmap al avance alcanzado

El roadmap es un plan que pueden revisar con criterio. **Pueden agregar un change intermedio, con alcance acotado, para completar o mejorar las funcionalidades que ya tienen**, sin implementar antes todos los changes pendientes. Lo importante es respetar las dependencias reales: una funcionalidad necesita sus requisitos técnicos y de negocio, aunque cambien su ubicación o numeración en el plan.

Ejemplo: su roadmap tiene 30 changes, completaron hasta el 10 y el frontend estaba previsto a partir del 15. Pueden proponer un nuevo change para construir una interfaz de las funcionalidades ya disponibles —por ejemplo, autenticación y consulta de agenda—, dejando fuera las pantallas y operaciones que dependan de backend aún no implementado. No necesitan completar los changes 11 a 14 si la interfaz acotada no depende de ellos.

1. **Revisen el estado real:** identifiquen qué funciona, qué está probado y qué dependencias faltan para el resultado que quieren mostrar.
2. **Definan un resultado completo y alcanzable:** una interfaz conectada a endpoints existentes, el cierre de una funcionalidad concreta o una mejora visual de las pantallas disponibles. Un alcance pequeño debe poder terminarse y comprobarse de principio a fin.
3. **Actualicen el roadmap:** agreguen el nuevo change con un identificador propio, propósito, alcance, exclusiones y dependencias. Expliquen qué parte adelanta, reemplaza o reduce respecto del plan original y ajusten las referencias de los changes futuros para evitar duplicaciones.
4. **Recorran el ciclo SDD completo:** propuesta, escenarios verificables, implementación, pruebas y archivado. Conserven la trazabilidad de los changes previos; no reescriban su historia para hacerlos aparecer como terminados.

También pueden planificar un change breve que aplique la skill propia o el MCP a una necesidad concreta del sistema. **La skill y el MCP pueden utilizarse en el mismo change o en changes distintos**, siempre que quede claro el aporte de cada uno. Estos nuevos changes cuentan para el mínimo si son relevantes, están completos, probados y archivados bajo los mismos criterios que los demás.

Reducir el alcance implica entregar íntegramente lo que se comprometieron a hacer. No significa archivar una funcionalidad incompleta, omitir dependencias necesarias o presentar una pantalla simulada como si estuviera integrada al sistema.

2. CUÁNDO CUENTA UN CHANGE

Un change cuenta cuando recorrió el ciclo SDD del proyecto: **propuesto, aplicado, verificado y archivado**. Debe poder relacionarse su especificación con el código, las pruebas y el resultado mostrado.

1. **Propuesto:** propósito, alcance, diseño, escenarios verificables y tareas definidos en los artefactos correspondientes.
2. **Aplicado:** implementación integrada al proyecto, con las tareas del alcance completadas.
3. **Verificado:** pruebas ejecutadas y resultado comprobado. Mantengan los tests automatizados correspondientes a sus escenarios y ejecútenlos antes del archivado; comprueben también el funcionamiento real de lo implementado.
4. **Archivado:** cierre registrado en OpenSpec y estado del roadmap actualizado.

Archivar no sustituye probar. Una carpeta en `archive/`, una casilla marcada o la afirmación del agente de que terminó no bastan si el resultado no funciona. Un change archivado con funcionalidades incompletas o sin la verificación que exige su alcance no se toma como completo para el mínimo.

Pueden existir changes pendientes o en desarrollo. Deben distinguirlos claramente de los terminados: **no se contabilizan ni se presentan como funcionalidades completas**. Si una limitación impide verificar una parte comprometida de un change, deben resolverla antes de contarlo como completo.

3. COMPRENDER Y DEMOSTRAR EL AVANCE

Para cada change presentado como completo, expliquen brevemente con palabras propias **qué problema resuelve, qué generó o modificó y cómo comprobaron el resultado**. No se espera memorizar el código: se espera conocer lo que se aceptó y archivó.

El sistema debe estar **levantado y funcionando hasta el alcance declarado**. Preparen el entorno y los datos de prueba antes de grabar para dedicar el tiempo del video a mostrar resultados.

Si el alcance incluye…	La demostración debe mostrar…
Fundación técnica	La estructura creada, para qué sirve, el arranque y las comprobaciones previstas en ese change. Mostrar carpetas vacías por sí solo no acredita una fundación ejecutable.
Modelos y persistencia	La estructura de datos y una verificación real acorde a sus escenarios: migración, creación, consulta o prueba de restricciones, según corresponda.
Backend y API	Backend en ejecución, Swagger abierto y solicitudes reales a los endpoints incluidos. Mostrar las respuestas y explicar qué confirman.
Autenticación en backend	Autenticación mediante la API en Swagger y comprobación del acceso protegido según el diseño implementado. No hace falta un formulario web si el change no lo incluye.
Frontend funcional	Frontend y backend levantados, con el flujo implementado: por ejemplo, iniciar sesión, consultar la agenda o gestionar un turno, según el alcance real.

Organicen una demostración continua que permita reconocer el resultado de los changes declarados. No es necesario ejecutar en el video cada caso de toda la suite: muestren el resultado de los tests y las operaciones representativas que evidencien el funcionamiento del alcance completo. Un frontend solo generado o una API que nunca se ejecutó no acreditan esas funcionalidades.

4. SKILL PROPIA Y SERVIDOR MCP

Retomen el trabajo de **MCP + Skills** y expliquen brevemente su integración al proyecto:

- **Skill propia:** qué necesidad resuelve, cómo la crearon —incluida la skill creadora utilizada—, cómo la adaptaron y dónde quedó integrada y referenciada para que el agente pueda utilizarla.
- **Servidor MCP:** cuál instalaron o conectaron, qué capacidades aporta y cómo comprobaron la conexión y el uso de una herramienta real.
- **Aplicación:** en qué change o changes utilizaron cada incorporación, qué solicitaron, qué resultado obtuvieron y cómo revisaron ese resultado. Puede tratarse de su preparación, implementación o verificación.

Muestren evidencia concreta del uso: solicitud, herramienta o procedimiento utilizado, resultado y aporte al change. Tener la skill y el MCP instalados o mostrar sus nombres en una lista no demuestra su aplicación. Una skill y un MCP son incorporaciones diferentes; no se exige desarrollar un servidor MCP propio ni utilizar servicios pagos.

La preparación de un change mediante estas herramientas puede mostrarse como evidencia de uso, pero **ese change solo cuenta para el mínimo cuando también está implementado, verificado y archivado**. Mantengan en el repositorio la skill, sus referencias y las instrucciones reproducibles del MCP, sin credenciales.

Ejemplo orientativo: una mejora visual con skill y MCP

Si ya tienen una interfaz, pueden proponer un change para mejorar de manera coherente las páginas incluidas en su alcance: navegación, jerarquía visual, formularios, mensajes y adaptación a distintos tamaños de pantalla. Si todavía no tienen frontend, pueden crear uno acotado a las funcionalidades disponibles, como se explicó en el apartado del roadmap.

Para ese trabajo podrían investigar referencias de skills de diseño, crear y adaptar una skill propia que guíe las mejoras, y utilizar una herramienta de un MCP de diseño compatible con su agente. Otra opción es aplicar su skill y su MCP a una funcionalidad breve diferente. **El diseño visual es una alternativa, no un requisito adicional del TPI.** No se exige una herramienta, proveedor ni servicio pago en particular.

El siguiente prompt es un punto de partida para conversar con el agente. Adapten el objetivo y las referencias a su proyecto antes de utilizarlo:

```
Revisá el estado real de nuestro sistema odontológico y su roadmap.
Queremos planificar un change para [crear una interfaz acotada / mejorar
el apartado visual existente] de las funcionalidades ya implementadas.

Primero identificá las páginas o flujos alcanzados, los endpoints
disponibles y las dependencias pendientes. No des por terminado el
roadmap ni incluyas funcionalidades de backend que todavía no existen.

Investigá referencias de skills de diseño, como Impeccable o
frontend-design, contrastando sus fuentes. Con ayuda de una skill
creadora, proponé una skill propia adaptada a nuestro proyecto:
propósito, procedimiento y criterios de revisión.

Evaluá un MCP de diseño compatible con nuestro agente; por ejemplo,
una integración disponible para Figma o Stitch. Verificá su existencia,
capacidades, requisitos, permisos y costos antes de elegirlo. Si esa
opción no está disponible, proponé una alternativa accesible y útil.

Definí qué aportará la skill y qué herramienta real del MCP usaremos.
Agregá el change al roadmap con alcance, exclusiones, dependencias y
escenarios verificables. Si ya existe frontend, identificá todas las
páginas incluidas y planificá una mejora coherente para cada una.
Conservá el comportamiento que funciona y planificá cómo probar la
integración, los flujos y el resultado visual. Presentá la propuesta
para que el equipo la revise antes de implementar.
```

La investigación y las herramientas elegidas deben servir al resultado concreto. En el video expliquen qué adoptaron de las referencias, cómo quedó su skill propia, qué herramienta del MCP ejecutaron y qué cambió en el sistema. Si optaron por changes distintos, muestren la relación entre cada herramienta y el change en el que se utilizó.

5. VIDEO DE PRESENTACIÓN Y DEMOSTRACIÓN

Presenten **un video de hasta 10 minutos en total por equipo**. El límite incluye la explicación de los changes, la skill y el MCP, y la demostración del sistema. Todos los integrantes deben presentarse y hablar; mantengan cámara encendida al menos durante la presentación personal, como en el trabajo anterior.

Distribución sugerida —pueden ajustarla sin superar el total—:

Tiempo orientativo	Contenido
0:00–0:30	Presentación del equipo y alcance alcanzado.
0:30–3:00	Recorrido breve por los changes completos: propósito, resultado y verificación. Mostrar roadmap y archivado.
3:00–6:00	Skill propia y MCP: creación o conexión, integración y aplicación concreta.
6:00–10:00	Resultado de las pruebas y demostración del sistema funcionando hasta el punto alcanzado.

Compartan la explicación de manera que todos puedan evidenciar comprensión. Muestren la pantalla y el trabajo real. Las diapositivas, si las utilizan, acompañan la explicación; la demostración debe permitir ver la aplicación ejecutándose.

6. REPOSITORIO Y FORMATO DE ENTREGA

Se conserva el formato de entrega del Parcial 2: **un único archivo `.txt`** que contenga el enlace accesible al video, el enlace al repositorio actualizado, la cantidad de integrantes y el nombre y apellido de todos. Carguen el archivo en el espacio **Entrega TPI de su comisión: C1, C3 o C4**.

A diferencia del Parcial 2, el repositorio debe contener **todo el avance del sistema que presentan en el TPI**, no solamente el primer change. Deben conservarse la fundación, el roadmap actualizado, los artefactos SDD y changes archivados, el código, los tests, la skill propia y la documentación de configuración y uso del MCP.

El `README.md` debe permitir reproducir el alcance presentado: requisitos, instalación, configuración de ejemplo sin secretos, preparación de datos ficticios, arranque del backend y del frontend si corresponde, acceso a Swagger y ejecución de tests. Indiquen claramente qué está completo y qué queda pendiente.

Nombre sugerido: `TPI-C1-Apellido1-Apellido2-Apellido3.txt`. Reemplacen la comisión y conserven únicamente los apellidos que correspondan.

```
ENTREGA — TPI METODOLOGÍA I
Comisión: 1
Video explicativo: https://...
Repositorio del proyecto: https://...
Cantidad de integrantes: 3
Integrantes:
1. Nombre Apellido
2. Nombre Apellido
3. Nombre Apellido
```

Comprueben los permisos de ambos enlaces antes de entregar. El docente debe poder acceder al video y al repositorio. No dejen enlaces de ejemplo o campos sin completar. Las fechas y horarios se consultan en la tarea de entrega de cada comisión dentro del campus.

7. CRITERIOS DE EVALUACIÓN

- **Avance relevante y completo:** cumplimiento del mínimo grupal con changes que aporten al sistema y respeten sus dependencias.
- **Funcionamiento y pruebas:** resultados verificables, tests correspondientes ejecutados y demostración real acorde al alcance declarado.
- **Comprensión y trazabilidad:** explicación propia de lo realizado y coherencia entre roadmap, especificaciones, implementación y archivado.
- **Skill y MCP:** integración y uso con un aporte concreto al proyecto, explicados y evidenciados.
- **Comunicación y entrega:** participación de todos, video dentro del límite y archivo TXT con información completa y enlaces accesibles.

La cantidad de changes por encima del mínimo no reemplaza su calidad. Se valora un avance consistente y probado. No se exige completar todo el roadmap ni desarrollar frontend cuando todavía queda fuera del alcance presentado.

LISTA DE VERIFICACIÓN ANTES DE ENTREGAR

- El total de changes válidos alcanza al menos dos por integrante y contempla los ya completados en el Parcial 2.
- Cada change contado tiene su alcance completo, pruebas verificadas y archivado; los pendientes están identificados aparte.
- Si agregaron changes intermedios, el roadmap explica su alcance y sus dependencias, sin duplicar trabajo ni presentar funcionalidades pendientes como completas.
- El equipo sabe explicar qué hizo cada change y mostrar su resultado.
- El sistema arranca y permite ejecutar lo que se declara implementado.
- La skill propia y el MCP están integrados y tienen evidencia de aplicación a un change.
- Todos hablan en el video y su duración total no supera los 10 minutos.
- El TXT incluye video, repositorio, cantidad de integrantes y nombres completos, con acceso verificado.

CONSIDERACIONES Y CONCLUSIONES ESPERADAS

Utilicen datos ficticios de pacientes y profesionales. Mantengan claves, tokens y contraseñas fuera del repositorio, de la entrega y del video. Revisen los resultados del agente antes de aceptarlos: el equipo debe comprender y validar lo que presenta.

Al finalizar, deben poder mostrar cómo un proyecto fundado se convierte en un sistema que funciona mediante changes completos, explicar el valor de las herramientas incorporadas y reconocer con precisión el alcance logrado y lo que todavía falta.